Task Behavior Graphs: A Domain-Driven Model for Profiling Queue Worker Tasks under Uncertain Resource Demand

Anonymous Author(s)

Abstract

TODO

Keywords: queue workers; task profiling; workload characterization; resource demand; worker occupancy; behavior graphs; observability; instrumentation; admission control; overload regulation.

1 Introduction

Queue-based execution decouples work production from work execution. A producer submits a task instance to a queue, and one or more workers later reserve, execute, retry, acknowledge, or fail that work. Although this pattern is often presented as a background-job abstraction, production queues frequently execute operationally significant work: replication, repair, cache rebuilds, imports, media processing, external-service calls, validation, notification delivery, and other asynchronous operations. In such systems, a worker is an execution boundary rather than merely a message consumer.

This paper studies profiling at the level of logical task attempts inside queue-worker systems. This level is narrower than a worker process, container, node, or queue, but broader than an individual function call or trace span. It is the level at which a concrete execution of a queued task consumes resources, holds bounded execution capacity, produces outcomes, and may be retried or failed. Coarser observations are useful, but they do not by themselves explain which task kind or attempt produced the observed behavior.

The central difficulty is that task behavior is conditional. A task kind and a payload-size bucket may provide a useful baseline, but they do not fully determine resource demand or worker occupancy. Two attempts of the same replication task kind may process similar payloads and still differ because one runs against healthy peers while another runs under degraded peer health, unstable network paths, disk pressure, or retry backoff. The task kind has not changed; the execution context has changed, and that context may affect only selected cost dimensions.

This paper uses a role-specific view of profiling. Task identity and intrinsic features describe the requested work. Execution context describes input-side state around an attempt. Active resource demand and worker occupancy are attempt-attributed cost-side estimates. Observations provide evidence. Uncertainty, risk, and profiling policy are downstream uses of estimates, residuals, and consequences. These roles are related, but they are not interchangeable.

Existing work motivates parts of this view. Cluster managers and fairness policies reason about multiple resource dimensions and observed usage [6, 11]. Workload studies show heterogeneity, dynamic behavior, and task-usage variation at scale [1, 9, 12]. Workload classification systems infer behavior rather than relying only on static resource assumptions [4, 5]. Large distributed systems are often shaped by tail behavior rather than average case alone [2]. Observability systems provide essential metrics, traces, logs, and profiles, but they introduce overhead and require budgeted sampling or instrumentation policies [7, 8, 10]. These lines of work do not remove the need for a queue-worker-specific model that connects logical tasks, execution context, cost estimates, uncertainty, and profiling policy.

This paper proposes *Task Behavior Graphs* as that model. A Task Behavior Graph is a task-kind- or task-family-level representation that connects canonical features to cost, uncertainty, and risk-relevant outputs through explicit influence assumptions. The graph is not treated as proof of causality. It is a structured modeling artifact: it exposes which features are believed to matter, which cost dimensions they influence, and where the model remains uncertain.

The paper makes five contributions. First, it frames queue-worker task profiling as conditional behavior estimation rather than raw telemetry collection. Second, it separates intrinsic task features, execution context, active resource demand, worker occupancy, uncertainty, and operational risk into distinct model roles. Third, it introduces Task Behavior Graphs as a representation for connecting these roles at task-kind or task-family granularity. Fourth, it describes how raw observations can be mapped into canonical features and observed output values through descriptors, manifests, and domain-specific extensions. Fifth, it outlines how profiling policy can be selected from task behavior, uncertainty, consequence, and instrumentation budget.

The remainder of the paper is organized as follows. Section 2 defines the core terminology and notation. Section 3 provides the technical background and motivating distinctions. Section 4 states the profiling problem. Sections 5–8 introduce the cost model, influence domains, task taxonomy, and graph representation. Sections 9–13 discuss feature mapping, profiling methods, observation points, uncertainty, risk, and policy selection. Section 14 describes manifests and domain packs. The remaining sections discuss case studies, evaluation, limitations, related work, and conclusions.

2 Core Terminology and Notation

This section defines only the core terms and notation required to state the problem and introduce the model. The paper uses a larger terminology set, but the complete reference glossary is deferred to Appendix A. The main text therefore avoids front-loading lifecycle details, implementation terms, and domain-specific vocabulary before the problem is stated.

The terminology is role-specific. A term describes one role in the model: identity, intrinsic work, execution context, observation, canonical feature, resource demand, worker occupancy, uncertainty, risk, or policy. A single operational event may connect several roles, but that connection does not make the roles equivalent. Section 3 explains the most common confusions; this section fixes the vocabulary and notation used by the remaining sections.

2.1 Core Terms

Table 1: Core terms used in the main model.

Term	Definition
task	A generic unit of work handled by a queue-worker system. The term is used only when the distinction between task kind, task instance, and task attempt is not important.
task kind	The semantic type of work submitted to a queue. A task kind describes what work is requested and selects reusable model structure, but it does not fully determine resource demand, worker occupancy, or risk under every execution context.
task instance	A concrete logical unit of work placed into a queue. A task instance belongs to a task kind and may produce one or more execution attempts.

Term	Definition
task attempt	One concrete execution of a task instance. Retries, lease expiration, worker failure, cancellation, or rescheduling may cause a single task instance to have multiple attempts. Resource usage, worker occupancy, observations, outcomes, and residuals are usually measured or estimated at attempt granularity.
worker	An executor that dequeues, reserves, executes, retries, acknowledges, discards, or fails task attempts. In this paper, a worker is treated as an execution boundary rather than merely a message consumer.
task behavior	The observed or estimated execution pattern of a task attempt or task kind under a given execution context, including resource demand, worker occupancy, outcome, uncertainty, and risk-relevant effects.
execution context	The input-side environment and state under which a task attempt executes. Execution context includes worker state, node pressure, dependency health, network condition, disk pressure, cache state, tenant pressure, retry state, and other runtime or domain-specific conditions. It may influence cost, but it is not itself a cost output.
observation	Any measurement, event, trace, profile sample, counter, log entry, or domain-specific signal exposed by a queue, worker runtime, operating system, container runtime, dependency, tracing system, or domain component.
canonical feature	A typed and normalized model input derived from task metadata, execution context, raw observations, or signals. Task Behavior Graphs operate over canonical features rather than arbitrary raw metrics.
influence domain	A semantic role assigned to a feature, observation, output, or model concept. Influence domains separate task identity, intrinsic demand, execution context, resource demand, worker occupancy, outcome, uncertainty, risk, and profiling observations.
cost dimension	A typed dimension of attempt-attributed cost. A cost dimension has a role, unit, value domain, attribution boundary, observation semantics, and composition semantics.
resource demand	The modeled active resource need expected for a task attempt along a resource dimension, such as CPU time, memory peak, disk I/O, network I/O, runtime allocation, or accelerator usage. Resource demand is distinct from observed resource usage and from environment-side resource pressure.
resource usage	The observed attempt-attributed use of a resource during execution. Resource usage is measured after or during execution and is distinct from estimated resource demand.

Term	Definition
worker occupancy	Bounded execution capacity held by a task attempt over time, such as worker-slot time, lease time, connection hold time, lock hold time, semaphore hold time, or tenant-level concurrency time. Occupancy exists only for intervals during which the corresponding capacity is assigned to the attempt.
worker-slot time	The time during which a task attempt holds a worker execution slot. This is the core worker-occupancy dimension in queue-worker systems.
uncertainty	Unreliability or unpredictability of a task behavior estimate. Uncertainty may come from missing features, hidden state, measurement error, drift, heavy tails, multimodality, or insufficient history.
operational risk	The consequence-weighted danger of underestimating, misclassifying, or mishandling a task attempt. Operational risk is modeled separately from task cost.
Task Behavior Graph	A typed graph associated with a task kind or task family. It connects canonical features across influence domains and supports estimation of resource demand, worker occupancy, uncertainty, and operational risk. A Task Behavior Graph makes modeled influence assumptions explicit; it is not assumed to prove causality by itself.
profiling policy	The selected observation and instrumentation strategy for a task kind, task attempt, or execution context. A profiling policy may choose minimal accounting, detailed accounting, tracing, sampling, tail-aware profiling, diagnostic profiling, or isolated diagnostic execution.

2.2 Notation

Table 2: Notation used throughout the paper.

Symbol	Meaning
$\mathcal{K}$	The set of task kinds.
$\mathcal{I}$	The set of task instances.
$\mathcal{A}$	The set of task attempts.
$i \in \mathcal{I}$	A task instance.
$a \in \mathcal{A}$	A task attempt.
$k(i) \in \mathcal{K}$	The task kind of task instance i.
$i(a) \in \mathcal{I}$	The task instance executed by attempt a.
k_a	A shorthand for $k(i(a))$, the task kind of attempt a.

Symbol	Meaning
$\mathcal{D}$	The set of influence domains.
$\mathcal{F}$	The set of canonical features.
z_a	The canonical feature vector available for attempt a after feature mapping and normalization. The vector may be partial and may include explicit missingness indicators.
P_X	The projection from z_a to intrinsic task features.
P_C	The projection from z_a to execution-context features.
$\mathbf{x}_a$	The intrinsic-feature projection $P_X(z_a)$ for attempt a.
$\mathbf{c}_a$	The execution-context projection $P_C(z_a)$ for attempt a.
θ_k	Task-kind-specific model state for task kind k, including declared descriptors, learned parameters, historical summaries, calibration data, quantile sketches, residual statistics, or domain rules.
F_k	The task-kind-specific cost estimator for task kind k.
$\mathcal{R}$	The set of active resource-demand dimensions.
$\mathcal{O}$	The set of worker-occupancy dimensions.
$\mathcal{Q}$	The typed disjoint union of cost dimensions, $\mathcal{Q} = \mathcal{R} \dot{\cup} \mathcal{O}$.
$\mathcal{Q}_k$	The subset of cost dimensions relevant to task kind k.
$q \in \mathcal{Q}$	A typed cost dimension, either a resource-demand dimension or a worker-occupancy dimension.
Δ_q	The descriptor for cost dimension q, including role, unit, value domain, attribution boundary, observation semantics, and composition semantics.
ρ_q	The role of cost dimension q: resource demand or worker occupancy.
u_q	The unit of cost dimension q.
$\mathcal{V}_q$	The value domain of cost dimension q.
α_q	The attribution boundary for cost dimension q.
ω_q	The observation semantics for cost dimension q.
κ_q	The composition semantics or composition operator for cost dimension q.
$\widehat{D}_r(a)$	Estimated resource demand for resource dimension $r \in \mathcal{R}$ during attempt a.
$\widehat{U}_o(a)$	Estimated worker occupancy for occupancy dimension $o \in \mathcal{O}$ during attempt a.
$\widehat{\mathbf{D}}(a)$	The resource-demand estimate vector for attempt a.
$\widehat{\mathbf{U}}(a)$	The worker-occupancy estimate vector for attempt a.

Symbol	Meaning
$\widehat{\mathbf{C}}(a)$	The structured attempt-level cost estimate, represented as $(\widehat{\mathbf{D}}(a), \widehat{\mathbf{U}}(a))$.
$F_{k,q}$	The dimension-specific estimator for task kind k and cost dimension q.
$\widehat{Y}_q(a)$	The estimated value for cost dimension $q \in \mathcal{Q}_{k_a}$ during attempt a. This is a dimension-generic notation used when the same expression applies to both resource-demand and worker-occupancy dimensions.
$Y_q(a)$	The observed attempt-attributed value for cost dimension q, when such an observation is available. For $q \in \mathcal{R}$ this is observed resource usage; for $q \in \mathcal{O}$ this is observed worker occupancy.
$\widehat{B}_q$	The estimated intrinsic baseline function for cost dimension q.
$\widehat{A}_q$	The estimated contextual amplification factor for cost dimension q.
$\widehat{O}_q$	The estimated additive overhead function for cost dimension q.
$\epsilon_q(a)$	Residual error for cost dimension q, defined when $Y_q(a)$ is available as $Y_q(a) - \widehat{Y}_q(a)$.
$\widehat{Y}_{q,p}(a)$	The estimated p-quantile or percentile-level value for cost dimension q during attempt a.
$\mathrm{Risk}(a)$	The operational-risk estimate for attempt a.
$\mathcal{G}_k$	The Task Behavior Graph associated with task kind k.
V_k	The set of nodes in Task Behavior Graph $\mathcal{G}_k$.
E_k	The set of edges in Task Behavior Graph $\mathcal{G}_k$.
$v \in V_k$	A node in Task Behavior Graph $\mathcal{G}_k$.
$e \in E_k$	An edge in Task Behavior Graph $\mathcal{G}_k$.
$\delta(v)$	The influence domain assigned to graph node v.
w_e	The estimated sensitivity or weight associated with graph edge e.
γ_e	The confidence value associated with graph edge e.

3 Background and Motivation

This section provides the technical background needed for the problem statement. It does not introduce the full model. Its purpose is to explain why queue-worker task profiling cannot be reduced to process-level monitoring, queue-level metrics, or generic tracing. The section first identifies the task attempt as the relevant execution unit. It then separates the roles that are most often conflated: attempt-side resource demand, attempt-side worker occupancy,

environment-side execution context, observed symptoms, and downstream risk. Finally, it motivates feature mapping and budgeted profiling.

3.1 Queue-Worker Execution and Task Attempts

Queue-worker systems execute work through a lifecycle that usually includes enqueue, reserve or dequeue, start, processing, retry or backoff when needed, commit or output write, and acknowledgement or failure. A single task instance can produce multiple attempts because of retry policy, lease expiration, timeout, cancellation, worker failure, or rescheduling. Each attempt may execute under a different worker state, dependency state, retry state, or domain state.

The attempt is therefore the most precise unit for attributing execution behavior. A task kind describes the semantic type of requested work. A task instance describes one queued unit of that kind. A task attempt describes one concrete execution of that instance under one execution context. Resource usage, worker occupancy, outcome, and residual error are usually measured or estimated most cleanly at this boundary. Aggregating only by queue, worker process, or node can reveal pressure, but it can hide which task kind, attempt, or context produced the observed behavior.

A worker may hold capacities that are not visible as high active utilization. During an attempt it may hold a worker slot, task lease, dependency connection, lock, semaphore permit, or tenant-level concurrency slot. These held capacities are occupancy outputs attributed to the attempt. They are different from the surrounding worker or worker-pool state. For example, a worker pool may already be close to saturation before an attempt starts; that pool state is execution context. If the attempt then holds one worker slot for thirty seconds, that duration is worker-slot occupancy attributed to the attempt.

3.2 Resource Demand, Worker Occupancy, and Context Pressure

Distributed resource-management systems commonly reason about multiple resource dimensions rather than a single scalar load. Cluster managers and allocation policies account for dimensions such as CPU, memory, storage, and other capacity constraints when placing, admitting, or allocating work [6, 11]. Queue-worker profiling inherits this multi-resource setting, but adds a task-attribution problem: the model must distinguish active resource demand, worker occupancy, and context pressure.

Active resource demand describes the modeled active resource need of the attempt itself, such as CPU time, memory peak, allocation volume, disk reads and writes, network transfer volume, or accelerator use. Worker occupancy describes bounded execution capacity held by the attempt over time, such as worker-slot time, lease time, connection hold time, lock hold time, semaphore hold time, or tenant concurrency time. Context pressure describes the surrounding state that may influence the attempt, such as CPU saturation, memory pressure, disk queue depth, network instability, dependency saturation, connection-pool pressure, or worker-pool utilization.

These roles are related by influence, not by identity. A degraded dependency may increase dependency wait time. That wait may increase worker-slot time if the slot remains held during the wait. The same degradation may also increase retry count or wall-clock duration. In this chain, dependency health is a context factor, dependency wait time is an observed intermediate duration, retry count and wall-clock duration are symptoms or outcomes, and worker-slot time is an occupancy cost only when the slot remains held. Treating all of them as the same quantity would obscure attribution and double-count one underlying degradation.

Demand must also be separated from pressure. CPU demand is not CPU pressure; disk I/O demand is not disk pressure; network transfer volume is not network instability; worker-slot time is not worker-pool pressure. Demand and occupancy are attempt-attributed quantities. Pressure

is an input-side state of the environment. A profiling model should preserve these roles before projecting them into any score used by a scheduler, admission controller, or profiling policy.

3.3 Role Assignment for Observations

Observations do not carry model roles by themselves. A raw metric, span, log record, profile sample, or domain event may become an input feature, an observed output value, a symptom, an outcome, or risk evidence depending on what it measures and how it is attributed.

For example, a packet-loss metric may be mapped to a network-instability context feature. Worker start and finish events may be mapped to observed worker-slot time if they delimit the interval during which a worker slot is held. A timeout log may be treated as a symptom or outcome of a dependency call. A sustained increase in queue delay may be risk-relevant evidence. These mappings are semantic choices; they are not implied by metric names alone.

This role assignment prevents double counting. Network degradation, dependency wait, retry count, wall-clock duration, and worker-slot time may all co-occur in the same incident. A model should not treat all of them as independent root causes. It should represent which signal describes context, which signal is an intermediate symptom, which signal is an observed cost dimension, and which downstream effect is a risk-relevant consequence.

3.4 Workload Variability and Context Dependence

Production workloads are heterogeneous and dynamic. Prior studies of large compute clusters and co-located workloads show substantial variation in task usage, infrastructure conditions, and workload behavior across time, machines, and task populations [1, 9, 12]. Workload classification systems such as Paragon and Quasar similarly motivate inferring behavior from observed properties and performance constraints rather than relying only on static declarations or user-provided reservations [4, 5]. For queue workers, the implication is direct: task kind, queue name, and declared size can provide a baseline, but not a complete behavior model.

A replication task illustrates the issue. Two attempts may have the same task kind and similar payload size, but execute against different replica sets. Under stable network conditions and healthy peers, the attempt may complete near its baseline. Under degraded peer health, packet loss, high replication lag, disk pressure, or retry backoff, the same logical operation may produce larger worker-slot time or connection hold time. The task did not become a different task kind. Its execution context changed, and that context amplified selected cost dimensions.

Other task families expose different sensitivities. A transformation task may be mostly shaped by record count, algorithm mode, CPU pressure, allocation behavior, and memory pressure. An external-service task may be shaped by dependency health, fan-out width, timeout policy, connection-pool availability, and retry state. A repair, compaction, cache rebuild, or migration task may be shaped by hidden domain state such as fragmentation, dirty data volume, replica divergence, cache warmth, or disk queue depth. A useful profiling model must therefore ask which features are relevant for a task kind rather than impose one global metric set on all tasks.

Average behavior is not enough. Large-scale distributed systems are often shaped by tail behavior and stragglers [2, 3]. In a queue-worker system, rare attempts that hold worker slots, leases, locks, or dependency connections for unusually long periods can reduce effective capacity and increase queueing delay even when mean duration appears acceptable. This motivates estimates that preserve distributional behavior when tails affect control decisions.

3.5 Profiling Overhead and Observation Policy

Observation has cost. Instrumentation, tracing, runtime profiling, high-cardinality labels, sampling decisions, and telemetry export can add CPU work, allocations, memory pressure,

storage volume, and network traffic. Dapper was designed with low overhead as a central concern, and OpenTelemetry treats sampling and telemetry performance as operational constraints [7,8,10]. Collecting more observations is therefore not automatically better.

The right observation strategy depends on the task kind, uncertainty, expected consequence, and instrumentation budget. Stable, low-consequence tasks may need only minimal accounting. Unknown, high-tail, context-sensitive, or high-consequence tasks may justify detailed accounting, sampled tracing, tail-aware capture, or diagnostic profiling. The instrumentation used to understand a system should not become an unbounded source of work or cardinality.

3.6 Motivating Gap

The preceding points expose a gap between available observations and the model needed for queue-worker task profiling. Resource-management systems reason about multiple resources; workload studies show variability and context dependence; observability systems expose metrics, traces, logs, and profiles. These pieces do not by themselves specify how to attribute behavior to logical task attempts, separate intrinsic demand from execution context, account for worker occupancy, assign semantic roles to observations, or choose instrumentation under an overhead budget.

The gap becomes clearer when tasks are compositional. A single task kind may include CPU processing, network transfer, dependency calls, retries, commits, and cleanup. Some quantities compose by summation, others by maximum, critical path, retry expansion, or occupancy-specific rules. Without an explicit model of metric roles and composition semantics, the system cannot reliably combine component behavior into an attempt-level estimate.

The next section states this gap as a profiling problem. Later sections define the cost model, influence domains, task taxonomy, Task Behavior Graphs, feature mapping, and profiling-policy selection as a structured response.

4 Problem Statement

The preceding section motivates a gap between available observations and the model needed to profile logical task attempts in queue-worker systems. This section states the problem addressed by the paper. It defines the system model, available information, expected outputs, main constraints, assumptions, and non-goals. The purpose is to state the problem without introducing the full model that later sections define.

At a high level, the problem is to estimate the behavior of a task attempt from its task identity, intrinsic task features, execution-context information, task-kind model state, and available history while respecting profiling overhead. The output is structured. It includes cost estimates, reliability diagnostics, risk interpretation, and a profiling-policy recommendation.

4.1 System Model

We consider queue-worker systems in which producers submit task instances to one or more queues and workers execute task attempts. A task kind identifies the semantic type of work requested. A task instance is one queued unit of that kind. A task attempt is one concrete execution of a task instance. A single task instance may produce multiple attempts because of retries, lease expiration, timeouts, cancellation, worker failure, or rescheduling.

The profiling boundary is the task attempt. This boundary is narrower than a worker process, container, node, or worker pool, but broader than a single function call or trace span. An attempt may include sequential, parallel, conditional, retried, or partially overlapping phases. The paper does not assume that every phase is perfectly instrumented.

Each task kind may have task-kind-specific model state. This state may contain declared descriptors, learned parameters, historical summaries, calibration data, quantile sketches, residual

statistics, or domain rules. The model state is updated or validated from historical observations, but the paper does not require one specific learning algorithm.

Each attempt executes under an execution context. Execution context is the input-side state surrounding the attempt; it is not the cost of the attempt. It may include worker state, worker-pool pressure, node pressure, dependency health, network condition, disk pressure, cache state, replica state, tenant pressure, retry state, and domain-specific hidden state. Some context factors are directly observable, some are derived from raw observations, and some remain hidden.

The model is task-kind-specific. Different task kinds may require different features, cost dimensions, observation points, composition rules, and profiling policies. A replication task may be sensitive to replica health and network condition. A compute-oriented transform may be more sensitive to record count, algorithm mode, CPU pressure, allocation behavior, and memory pressure. The problem therefore cannot assume one global feature set or one global metric list that is equally relevant to all task kinds.

4.2 Available Information

For an attempt a, the profiler may have access to five classes of information.

First, task identity describes what is being executed. It may include the task kind, task version, queue, producer, tenant, priority, deadline, and attempt number. Task identity selects the relevant task-kind-specific model; it is not by itself a cost estimate.

Second, intrinsic task features describe work contained in the task instance. Examples include payload size, record count, object count, partition size, batch size, input format, operation mode, declared resource hints, or domain metadata. These features may be available before execution and can provide a baseline for expected work.

Third, execution-context information describes the environment in which the attempt is executed or expected to execute. Examples include worker-pool pressure, node CPU or memory pressure, disk queue depth, network instability, dependency health, connection-pool pressure, cache warmth, replica health, replication lag, tenant quota usage, and retry state. These values are input-side conditions. They may influence resource demand or worker occupancy, but they are not themselves worker-occupancy values.

Fourth, task-kind model state describes reusable information associated with the task kind or task family. It may include descriptors, estimator parameters, default priors, historical distributions, residual statistics, confidence values, or domain-specified rules.

Fifth, historical observations describe previous attempts of the same or related task kinds. They may include observed resource usage, observed worker occupancy, durations, retry counts, outcomes, residuals, tail quantiles, or drift indicators. Historical data may be unavailable for new task kinds, new versions, rare contexts, or recently changed workloads. The problem must therefore support both calibrated and cold-start cases.

Raw observations are not assumed to be model-ready. Metrics, logs, traces, runtime profiles, and domain events must be interpreted, normalized, and mapped into canonical input features, observed output values, symptoms, outcomes, or risk evidence before the model can use them.

4.3 Desired Outputs

The profiling problem produces four classes of outputs.

The first output class is cost estimates. These include active resource-demand estimates over relevant resource dimensions and worker-occupancy estimates over relevant occupancy dimensions. Cost estimates may be point estimates, distributional estimates, or conservative envelopes.

The second output class is reliability diagnostics. These include residuals, uncertainty indicators, confidence values, drift indicators, missing-feature signals, insufficient-history indicators,

and tail-behavior summaries. These diagnostics describe how reliable the cost estimates are; they are not themselves active resource demand or worker occupancy.

The third output class is operational-risk interpretation. Risk is not the same as task cost. It captures the consequence of underestimating, misclassifying, or mishandling an attempt. Relevant risks may include queue starvation, retry amplification, SLO violation, replication lag, tenant impact, dependency overload, and cascading overload.

The fourth output class is a profiling-policy recommendation. Given the task kind, context, reliability diagnostics, risk, and profiling budget, the system should determine what observation strategy is justified. The policy output is a profiling decision, not a final scheduling, routing, or placement decision.

4.4 Core Challenges

The first challenge is conditional behavior. Attempts with the same task kind and similar intrinsic features may behave differently under different execution contexts. A model based only on task kind, queue, or payload bucket can provide a baseline, but it cannot explain context-dependent amplification, suppression, or drift.

The second challenge is multidimensionality. Active resource demand, bounded worker occupancy, reliability diagnostics, and operational risk have different meanings and units. Combining them prematurely into one scalar hides information needed by downstream control layers. A scalar score may be useful, but it should be a projection of a structured estimate rather than a replacement for it.

The third challenge is typed cost semantics. CPU time, memory peak, network bytes, worker-slot time, and lock hold time do not compose in the same way. Some dimensions sum, some take maxima, some are interval measures, and some follow critical-path or retry-expansion semantics. A profiling model must therefore treat cost dimensions as typed descriptors rather than as metric names.

The fourth challenge is attribution. Process-level, container-level, node-level, and queue-level observations are often easier to collect than task-attempt-level observations. Aggregate observations may indicate pressure, but they do not automatically identify which task kind, instance, attempt, phase, context factor, or dependency produced the observed behavior.

The fifth challenge is semantic interpretation. Raw observations do not state their model role. Without explicit semantic roles, a model may double-count related signals or treat symptoms as independent causes. The problem therefore requires a mapping from raw observations to canonical input features, observed output values, symptoms, outcomes, and risk evidence.

The sixth challenge is compositional behavior. A task kind may include CPU processing, network transfer, dependency calls, retries, commits, and cleanup. Some quantities compose by summation, some by maximum, some by critical path, and some by retry expansion or occupancy-specific rules. The profiling problem must preserve enough structure to combine component behavior consistently.

The seventh challenge is partial observability. Important influences may be missing, stale, hidden, or domain-specific. The model cannot assume that all relevant state is observable. Unexplained behavior should remain visible through residual error, reduced confidence, or a higher profiling level.

The eighth challenge is profiling overhead. Instrumentation is not free. Detailed tracing, runtime profiling, high-cardinality labels, frequent sampling, and telemetry export can affect the workload being measured. A profiling model must decide not only what would be useful to observe, but what is worth observing under an overhead budget.

4.5 Scope and Assumptions

This paper assumes that task attempts can be identified at least by task kind and attempt boundary. The model is less useful when all work is invisible inside one undifferentiated worker loop. The paper does not require perfect instrumentation, but it assumes that minimal accounting is possible for at least some attempts.

The paper assumes that some task metadata or intrinsic features may be available before or during execution. The exact features are domain-specific. A validation task, replication task, external API fan-out task, and media transformation task need not expose the same feature set.

The paper assumes that execution-context information may be available but incomplete. It may come from workers, nodes, queues, dependencies, telemetry systems, domain services, or historical profiles. The model must tolerate missing, stale, noisy, or partially aggregated context.

The paper assumes that profiling decisions are useful even when final scheduling, routing, placement, or admission decisions are made by another layer. The profiling model supplies structured estimates and observation policies. Downstream control layers may consume these outputs, but their full design is outside the scope of the paper.

The paper also assumes an open-world metric environment. No core system can know all possible domain-specific metrics in advance. The model must therefore allow raw observations to be mapped into canonical features and must keep missing or unexplained influences explicit instead of hiding them behind a fixed global metric list.

4.6 Non-Goals

This paper does not propose a complete distributed scheduler. It does not define a placement algorithm, routing algorithm, gossip protocol, load-balancing policy, or cluster-wide resource allocator. Such systems may use the estimates produced by task profiling, but they are not the object of this paper.

This paper does not replace metrics, logs, traces, runtime profiles, or existing observability tools. These systems provide observations. The problem addressed here is how to interpret observations for task-kind-specific profiling and how to decide which observations are justified.

This paper does not claim that Task Behavior Graphs prove causality. A graph can make modeled influence assumptions explicit, but causal validation requires additional evidence, experiments, interventions, or domain knowledge. The graph is a profiling model, not a causal proof.

This paper does not attempt to define a globally complete taxonomy of all workload metrics. It assumes that new domains can introduce new observations, features, and behavior components. Completeness is task-kind- and domain-specific rather than global.

This paper does not require exact prediction for arbitrary tasks. The goal is to produce structured estimates, expose uncertainty, and guide profiling policy. High residual error, drift, missing features, or high tail risk are valid model outputs, not failures to hide.

Finally, this paper does not specify a full production implementation of a package manager, expression language, runtime virtual machine, or cross-language manifest compiler. Later sections discuss manifests and implementation considerations only to the extent needed to make the proposed model concrete.

5 Task Cost Model

Section 4 states that task profiling should produce a structured estimate rather than a single scalar load value. This section defines the cost-side part of that estimate. The full profiling problem has several outputs: cost estimates, reliability diagnostics, risk interpretation, and a profiling-policy recommendation. This section models only the first output class: attempt-attributed cost.

In this paper, task cost has two primary output families: active resource demand and bounded worker occupancy. Execution overhead is not a third independent output family. It is modeled as a contribution to one or more resource-demand or worker-occupancy dimensions. Operational risk is not part of task cost; it is a downstream interpretation of cost estimates, uncertainty, consequences, and policy context.

The section proceeds from the unit of analysis to the dimensional model. It first defines attempt-level cost. It then introduces typed cost dimensions, task-kind-specific estimator state, dimension-specific estimator forms, observed values, residuals, and distributional estimates.

5.1 Cost as an Attempt-Level Quantity

The cost model is defined at task-attempt granularity. A task kind determines the reusable model structure, a task instance supplies intrinsic work features, and a task attempt is the concrete execution in which demand, occupancy, observations, outcomes, and residuals are realized.

Let $a \in \mathcal{A}$ be a task attempt. The attempt executes task instance $i(a) \in \mathcal{I}$, whose task kind is $k(i(a)) \in \mathcal{K}$. For readability, write

$$k_a = k(i(a)).$$

Let z_a be the canonical feature vector available for attempt a after feature mapping and normalization. The vector may be partial: missing values and unknown context should be represented explicitly rather than silently treated as zero. We use two role-specific projections:

$$\mathbf{x}_a = P_X(z_a), \qquad \mathbf{c}_a = P_C(z_a),$$

where $\mathbf{x}_a$ contains intrinsic task features and $\mathbf{c}_a$ contains execution-context features. Intrinsic features describe the work contained in the task instance, such as payload size, record count, object count, batch size, operation mode, or domain metadata. Execution-context features describe the environment and state around the attempt, such as worker state, node pressure, dependency health, network condition, disk pressure, cache state, retry state, tenant pressure, or domain-specific state.

For each task kind k, the model may maintain task-kind-specific state θ_k. This state may contain declared descriptors, learned parameters, historical summaries, calibration values, quantile sketches, residual statistics, or domain rules. The paper does not require one learning algorithm. The state θ_k represents the reusable information used to evaluate attempts of kind k.

The cost estimate for attempt a is produced by a task-kind-specific estimator:

$$\widehat{\mathbf{C}}(a) = F_{k_a}(\mathbf{x}_a, \mathbf{c}_a; \theta_{k_a}) .$$

The estimator returns a structured object rather than a scalar. It contains active resource-demand estimates and worker-occupancy estimates:

$$\widehat{\mathbf{C}}(a) = \Big(\widehat{\mathbf{D}}(a), \widehat{\mathbf{U}}(a)\Big) ,$$

where

$$\widehat{\mathbf{D}}(a) = \Big\{\widehat{D}_r(a) \mid r \in \mathcal{R}_{k_a}\Big\}, \qquad \widehat{\mathbf{U}}(a) = \Big\{\widehat{U}_o(a) \mid o \in \mathcal{O}_{k_a}\Big\} .$$

The sets $\mathcal{R}_{k_a}$ and $\mathcal{O}_{k_a}$ are the resource-demand and worker-occupancy dimensions relevant for task kind k_a.

A scalar may be derived from $\widehat{\mathbf{C}}(a)$ by a downstream policy, but such a scalar is not the cost model itself. For example, an admission policy or profiling-policy selector may apply a projection

$$s_\rho(a) = \pi_\rho\Big(\widehat{\mathbf{D}}(a), \widehat{\mathbf{U}}(a)\Big)$$

under policy ρ. The projection is policy-specific: it depends on the decision being made, the available capacity, and the relative importance assigned to dimensions. The cost model preserves the structured estimate because different dimensions have different units, attribution boundaries, observation semantics, and composition rules.

5.2 Typed Cost Dimensions

A cost dimension is not merely a metric name. It is a typed descriptor that states what is being estimated, how it is attributed to an attempt, how observed values are interpreted, and how component values compose.

Let

$$\mathcal{Q} = \mathcal{R} \dot{\cup} \mathcal{O}$$

be the typed disjoint union of resource-demand dimensions and worker-occupancy dimensions. A dimension $q \in \mathcal{Q}$ has a descriptor

$$\Delta_q = (\rho_q, u_q, \mathcal{V}_q, \alpha_q, \omega_q, \kappa_q) .$$

Here $\rho_q \in \{\text{resource}, \text{occupancy}\}$ is the role of the dimension, u_q is its unit, $\mathcal{V}_q$ is its value domain, α_q is its attribution boundary, ω_q is its observation semantics, and κ_q is its composition semantics.

The role ρ_q determines whether the dimension belongs to active resource demand or worker occupancy. The unit u_q may be seconds, bytes, operations, requests, slot-seconds, connection-seconds, lock-seconds, or another domain-specific unit. The value domain $\mathcal{V}_q$ is usually a non-negative subset of a unit-bearing numeric domain. The attribution boundary α_q specifies when an observed value belongs to attempt a. The observation semantics ω_q specifies how raw observations become an attempt-attributed observed value. The composition semantics κ_q specifies how component values combine.

For example, CPU time is a resource-demand dimension whose component values typically compose by summation over attributed execution intervals. Memory peak is a resource-demand dimension whose component values usually compose by maximum, not by summation. Worker-slot time is an occupancy dimension measured as slot-seconds over intervals during which a worker slot is held by the attempt. Connection hold time is an occupancy dimension measured over intervals during which a dependency connection is held by the attempt. These dimensions should not be forced into the same aggregation rule.

For a task kind k, the model uses a task-kind-specific subset

$$\mathcal{Q}_k \subseteq \mathcal{Q}.$$

The subset contains the cost dimensions relevant to that task kind. A CPU-oriented transformation may include CPU time, allocation volume, and memory peak. A replication task may include network output, disk reads, disk writes, worker-slot time, connection hold time, and lease time. The model does not assume that every task kind uses every dimension.

5.3 Resource Demand and Worker Occupancy

The two families of cost dimensions have different meanings. Resource demand is the modeled amount of active resource usage expected for an attempt. Observed resource usage is the measured attempt-attributed value when instrumentation provides such attribution. Worker occupancy is bounded execution capacity held by an attempt over time.

For $r \in \mathcal{R}_{k_a}$, $\widehat{D}_r(a)$ denotes the estimated resource demand of attempt a for resource dimension r. Resource-demand dimensions may include CPU time, memory peak, allocation volume, disk reads, disk writes, network input, network output, or accelerator usage. These dimensions describe active resource work associated with the attempt rather than the ambient state of the worker, node, dependency, or queue.

For $o \in \mathcal{O}_{k_a}$, $\widehat{U}_o(a)$ denotes the estimated worker occupancy of attempt a for occupancy dimension o. Occupancy exists only over intervals in which the corresponding bounded capacity is assigned to the attempt and cannot be used by another compatible attempt. Examples include

worker-slot time, lease time, connection hold time, lock hold time, semaphore hold time, and tenant-level concurrency time.

This distinction separates demand, occupancy, and pressure. Demand and occupancy are estimated or observed for the attempt. Pressure is a property of the execution context. CPU time is a resource-demand dimension; CPU saturation on the node is a context factor. Worker-slot time is occupancy attributed to an attempt; worker-pool pressure is the surrounding state of the pool. A context factor may influence a cost dimension, but it is not itself the cost dimension.

Waiting and elapsed time require the same care. Dependency health, network condition, connection-pool pressure, and retry state are context factors. Dependency wait time is an observed waiting signal. Wall-clock duration is an elapsed-time outcome. Worker-slot time and connection hold time are occupancy dimensions only when the corresponding bounded capacities remain held during the elapsed interval. A long dependency wait therefore increases worker-slot time only if the slot remains held.

A compute-dominant transformation may have high CPU time and allocation volume but release its worker slot quickly. Conversely, an external-service or replication attempt may have low CPU demand but high worker-slot time, connection hold time, or lease time when a dependency is slow, a peer is degraded, or retries introduce backoff. Active resource demand alone would miss the second cost pattern.

5.4 Dimension-Specific Estimators

Because cost dimensions have different units and composition semantics, the model does not require one universal formula for all dimensions. Instead, for each task kind k and dimension $q \in \mathcal{Q}_k$, the model defines a dimension-specific estimator

$$F_{k,q} : \mathcal{X}_k \times \mathcal{C}_k \times \Theta_k \to \mathcal{V}_q.$$

For attempt a, the point estimate for dimension q is

$$\widehat{Y}_q(a) = F_{k_a,q}(\mathbf{x}_a, \mathbf{c}_a; \theta_{k_a}) \, .$$

If $q \in \mathcal{R}_{k_a}$, then $\widehat{Y}_q(a)$ corresponds to a component of $\widehat{\mathbf{D}}(a)$. If $q \in \mathcal{O}_{k_a}$, then $\widehat{Y}_q(a)$ corresponds to a component of $\widehat{\mathbf{U}}(a)$.

A common estimator form decomposes a dimension into intrinsic baseline, contextual amplification, and additive overhead:

$$\widehat{Y}_q(a) = \kappa_q\Big(\widehat{B}_q(k_a, \mathbf{x}_a; \theta_{k_a}), \widehat{A}_q(k_a, \mathbf{x}_a, \mathbf{c}_a; \theta_{k_a}), \widehat{O}_q(k_a, \mathbf{x}_a, \mathbf{c}_a; \theta_{k_a})\Big) \, .$$

Here $\widehat{B}_q$ is the estimated intrinsic baseline, $\widehat{A}_q$ is the estimated contextual amplification factor, and $\widehat{O}_q$ is the estimated additive overhead contribution. The composition operator κ_q is taken from the dimension descriptor Δ_q.

For additive dimensions, a useful specialization is the affine form

$$\widehat{Y}_q(a) = \widehat{B}_q(k_a, \mathbf{x}_a; \theta_{k_a}) \cdot \widehat{A}_q(k_a, \mathbf{x}_a, \mathbf{c}_a; \theta_{k_a}) + \widehat{O}_q(k_a, \mathbf{x}_a, \mathbf{c}_a; \theta_{k_a}).$$

This affine form is not a universal law. It is appropriate only when the dimension's composition semantics supports proportional amplification plus additive contribution. CPU time or network-transfer duration may often use such a form. Memory peak may require maximum-based composition. Worker-slot time may require interval or critical-path semantics. Retry-sensitive dimensions may require retry expansion.

The terms have unit constraints. The estimate $\widehat{Y}_q(a)$, baseline $\widehat{B}_q$, and overhead $\widehat{O}_q$ are measured in the unit u_q. The amplification factor $\widehat{A}_q$ is dimensionless. In the usual cost setting, baseline and overhead are non-negative, and amplification is non-negative. Suppression is represented by amplification below one rather than by negative cost.

This decomposition is an engineering abstraction, not an independence claim. It does not assert that intrinsic work and execution context are statistically independent, nor does it require all dimensions to be linear. Its role is to separate three mechanisms: work implied by the task, contextual effects that modify that work, and additional overhead that is not well represented as a multiplier. The graph and manifest sections later make these mechanisms explicit for each task kind.

5.5 Execution Overhead and Profiling Overhead

The overhead term covers additional contribution to a cost dimension. It should not be confused with a separate output family. Overhead is represented in the unit and role of the dimension to which it contributes.

Execution overhead is overhead required by task execution itself. Examples include setup, deserialization, connection establishment, lease-extension bookkeeping, acknowledgement, commit, and cleanup. These effects may contribute to CPU time, allocation volume, disk writes, worker-slot time, connection hold time, or other dimensions depending on their attribution.

Profiling overhead is overhead introduced by observation policy. Examples include tracing, sampling, runtime profiling, high-cardinality label handling, telemetry export, and additional accounting. Profiling overhead can also contribute to cost dimensions when instrumentation is enabled. However, the decision to enable instrumentation belongs to the profiling-policy layer, not to the cost model. The cost model only represents the cost consequences once such instrumentation is part of the execution.

5.6 Observed Values and Residuals

The model distinguishes estimates from observed values. Before or during execution, the profiler produces $\widehat{Y}_q(a)$ for a cost dimension $q \in \mathcal{Q}_{k_a}$. After execution, instrumentation may provide an observed value $Y_q(a)$ for the same dimension. The value $Y_q(a)$ is not raw telemetry. It is an attempt-attributed observed value produced according to the dimension's observation semantics ω_q.

The residual is defined only when an observed value exists:

$$\epsilon_q(a) = Y_q(a) - \widehat{Y}_q(a), \qquad q \in \mathcal{Q}_{k_a}.$$

Absence of a residual does not imply model accuracy; it may simply mean that the dimension was not observed.

For $q \in \mathcal{R}_{k_a}$, $Y_q(a)$ is an attempt-attributed observed resource-usage value. For $q \in \mathcal{O}_{k_a}$, $Y_q(a)$ is an attempt-attributed observed occupancy value. Node-level CPU utilization may be useful context, but it is not observed CPU usage for a particular attempt. Wall-clock duration may provide evidence, but it becomes an occupancy value only for the capacity that remains held during the corresponding interval.

Residuals are signals about the relation between the model and the workload. A large residual may indicate missing intrinsic features, stale context, hidden dependency state, an incorrect task-kind classification, model misspecification, measurement error, or a tail event. A small residual does not prove causal correctness; it only indicates that the estimate was close to the observed value for that attempt and dimension.

Residuals connect the cost model to later uncertainty analysis. If residuals are large, systematic, or concentrated in particular contexts, the model may need additional features, a revised graph, a different taxonomy, or a higher profiling level. This section defines residuals only as differences between observed and estimated cost dimensions; later sections use them to reason about confidence, drift, and risk.

5.7 Distributional Cost Estimates

Attempt-level cost is often variable even within the same task kind and feature bucket. A single mean estimate can hide behavior that matters operationally, especially for tasks with retries, dependency waits, lock contention, cache misses, or degraded peers. The cost model therefore allows each dimension to be estimated as a distribution or as a set of summary statistics rather than only as a point value.

For each task kind k and dimension $q \in \mathcal{Q}_k$, the task-kind model state θ_k may contain distributional information, such as empirical histograms, quantile sketches, parametric distribution parameters, conservative envelopes, or prediction intervals. For quantile level $p \in (0, 1)$, write

$$\widehat{Y}_{q,p}(a)$$

for a quantile estimate of dimension q for attempt a. For example, $\widehat{Y}_{q,0.50}(a)$ may represent a median estimate and $\widehat{Y}_{q,0.99}(a)$ may represent a high-tail estimate.

Different decisions depend on different parts of the distribution. A mean CPU-time estimate may be sufficient for coarse accounting of stable compute tasks. A high-percentile worker-slot estimate may be more relevant for capacity protection when rare attempts can hold workers for long periods. A high-percentile connection-hold estimate may be more relevant for dependency protection than average network bytes.

Distributional estimates also prevent a common modeling error: treating variability as noise around one expected value. In queue-worker systems, variability may reflect distinct modes. A task may be fast when cache is warm and slow when cache is cold. A replication attempt may be short when peers are healthy and long when retries occur. An external-service attempt may be bimodal because dependency timeouts create a second latency mode. The model should allow such behavior to remain visible rather than compress it into one average.

This section does not define entropy, confidence, or risk metrics. It only states that cost estimates may be distributional. Later sections may derive uncertainty indicators from residuals, tail ratios, drift, missing features, multimodality, or insufficient sample size.

5.8 Boundaries of the Cost Model

The cost model defines what is estimated at attempt granularity: active resource demand and worker occupancy. It does not define the complete set of influence domains, task behavior classes, graph topology, raw-observation mapping, uncertainty metrics, risk model, or profiling-policy selection algorithm.

This boundary is important. Execution context provides input-side conditions that may influence cost; it is not itself a cost output. Symptoms and outcomes may provide evidence about cost; they are not automatically cost dimensions. Risk consumes cost estimates and consequence information; it is not part of the cost object. Profiling policy may use cost estimates and may introduce profiling overhead; the decision to enable instrumentation is not made by the cost model.

The next section introduces influence domains, which assign semantic roles to the inputs, outputs, symptoms, uncertainty indicators, risks, and profiling observations used by the rest of the paper.

6 Influence Domains

7 Task Taxonomy

8 Task Behavior Graphs

9 Metrics, Signals, and Features

10 Profiling Methods

11 Observation Points in Queue Workers

12 Entropy, Confidence, and Risk

13 Profiling Policy Selection

14 Task Behavior Manifests

15 Case Studies

16 Evaluation Methodology

17 Discussion

18 Threats to Validity

19 Related Work

20 Conclusion

References

[1] Yue Cheng, Ali Anwar, and Xuejing Duan. Analyzing alibaba's co-located datacenter workloads. In *2018 IEEE International Conference on Big Data*, Big Data '18. IEEE, 2018.

[2] Jeffrey Dean and Luiz André Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, 2013.

[3] Jeffrey Dean and Sanjay Ghemawat. Mapreduce: Simplified data processing on large clusters. In *Proceedings of the 6th Symposium on Operating Systems Design and Implementation*, OSDI '04, pages 137–150. USENIX Association, 2004.

[4] Christina Delimitrou and Christos Kozyrakis. Paragon: Qos-aware scheduling for heterogeneous datacenters. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '13. ACM, 2013.

[5] Christina Delimitrou and Christos Kozyrakis. Quasar: Resource-efficient and qos-aware cluster management. In *Proceedings of the Nineteenth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '14. ACM, 2014.

[6] Ali Ghodsi, Matei Zaharia, Benjamin Hindman, Andy Konwinski, Scott Shenker, and Ion Stoica. Dominant resource fairness: Fair allocation of multiple resource types. In *Proceedings of the 8th USENIX Symposium on Networked Systems Design and Implementation*, NSDI '11. USENIX Association, 2011.

[7] OpenTelemetry. Opentelemetry documentation: Sampling. `https://opentelemetry.io/docs/concepts/sampling/`, 2025. Accessed 2026-05-03.

[8] OpenTelemetry. Opentelemetry documentation: Performance. `https://opentelemetry.io/docs/zero-code/java/agent/performance/`, 2026. Accessed 2026-05-03.

[9] Charles Reiss, Alexey Tumanov, Gregory R. Ganger, Randy H. Katz, and Michael A. Kozuch. Heterogeneity and dynamicity of clouds at scale: Google trace analysis. In *Proceedings of the Third ACM Symposium on Cloud Computing*, SoCC '12. ACM, 2012.

[10] Benjamin H. Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal, Donald Beaver, Saul Jaspan, and Chandan Shanbhag. Dapper, a large-scale distributed systems tracing infrastructure. Technical report, Google, 2010.

[11] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at google with borg. In *Proceedings of the Tenth European Conference on Computer Systems*, EuroSys '15. ACM, 2015.

[12] Qi Zhang, Joseph L. Hellerstein, and Raouf Boutaba. Characterizing task usage shapes in google compute clusters. In *Proceedings of the 5th International Workshop on Large Scale Distributed Systems and Middleware*, LADIS '11. ACM, 2011.

A Extended Terminology

This appendix provides the complete terminology reference used by the paper. Section 2 intentionally keeps only the core terms needed for the main model. The extended glossary is organized by semantic area so that the main text can introduce concepts progressively while still keeping a precise reference for task entities, behavior classes, execution context, observations, influence domains, resource demand, worker occupancy, uncertainty, risk, graph structure, profiling policy, manifests, and queue-worker lifecycle stages.

A.1 Task Entities

Table 3: Task entity terms.

Term	Definition
task	A generic unit of work handled by a queue-worker system. The term is used only when the distinction between task kind, task instance, and task attempt is not important.
task kind	The semantic type of work submitted to a queue, such as copying a replication partition, rebuilding a cache entry, resizing an image, repairing a database record, dispatching a webhook, or validating a document. A task kind describes what work is requested, but it does not fully determine how that work behaves under every execution context.

Term	Definition
task instance	A concrete logical unit of work placed into a queue. A task instance belongs to a task kind and may produce one or more execution attempts.
task attempt	One concrete execution attempt of a task instance. Retries, lease expiration, worker failure, cancellation, or rescheduling may cause a single task instance to have multiple attempts. Resource demand, worker occupancy, observations, outcomes, and residuals are usually measured or estimated at attempt granularity.
task family	A group of task kinds that share a similar behavior-graph structure or profiling model. A task family can be used when one graph template applies to several related task kinds.
task identity	The identifying metadata of a task, including task kind, task version, queue, producer, tenant, priority, and deadline. Task identity identifies the work but does not itself define the cost of executing it.
task version	The version of a task kind, handler contract, payload schema, or behavior manifest. Task versions are useful for compatibility between historical profiles, manifests, and schema versions.
queue	The queue through which a task instance is submitted, stored, reserved, and eventually consumed by a worker. A queue name may provide useful context, but it is not a complete behavior model.
producer	The component that creates and submits a task instance to a queue. The producer may provide metadata or declared hints, but it does not fully determine the execution context.
worker	An executor that dequeues, reserves, executes, retries, acknowledges, discards, or fails task attempts. In this paper, a worker is treated as an execution boundary rather than merely a message consumer.
worker pool	A set of workers that execute task attempts. Worker-pool capacity and worker-pool occupancy are central to queue-worker profiling.
tenant	A user, customer, namespace, application, or logical group that consumes execution capacity. Tenants are relevant for isolation, fairness, consequence analysis, and operational risk.
tenant class	A category of tenants based on SLO, priority, plan, criticality, or operational risk. A tenant class is distinct from a task kind.

Term	Definition
priority	A scheduling or ordering attribute assigned to a task instance. Priority may influence profiling or admission decisions, but it is not itself resource demand or operational risk.
deadline	A completion target or time constraint associated with a task instance. A deadline may affect consequence and risk, but it is distinct from observed latency.

A.2 Task Behavior

Table 4: Task behavior terms.

Term	Definition
task behavior	The observed or estimated execution pattern of a task kind, task instance, or task attempt under a particular execution context. Task behavior includes resource demand, worker occupancy, retry behavior, dependency wait, outcome, uncertainty, and risk-relevant effects.
behavior profile	A summarized description of how a task kind behaves under a class of execution contexts. A behavior profile may include demand distributions, occupancy distributions, dominant behavior classes, uncertainty, risk class, and relevant features.
behavior class	An analytical category derived from observed or estimated behavior. Examples include compute-dominant, dependency-bound, retry-amplifying, worker-slot-heavy, payload-sensitive, state-dependent, or high-tail-risk behavior.
behavior mode	A context-dependent mode of behavior for a task kind. The term emphasizes that the same task kind may behave differently under different execution contexts.
compute-dominant behavior	Behavior in which CPU demand is the dominant active resource dimension. A task kind may exhibit compute-dominant behavior without being intrinsically a CPU task in all contexts.
memory-pressure-sensitive behavior	Behavior that is sensitive to memory pressure, allocation rate, working-set size, or garbage-collection effects. This behavior is not necessarily identical to being memory-bound.
disk-IO-dominant behavior	Behavior in which disk reads, disk writes, storage throughput, or I/O wait dominates execution. Disk I/O demand is distinct from disk pressure in the execution environment.

Term	Definition
network-sensitive behavior	Behavior that changes substantially with network condition, such as packet loss, jitter, retransmissions, bandwidth changes, or timeouts.
dependency-bound behavior	Behavior constrained primarily by external dependencies, such as databases, APIs, remote services, rate limits, quotas, or connection pools.
retry-amplifying behavior	Behavior in which retries substantially increase work, worker occupancy, or operational risk. Retry count may be a symptom or outcome rather than an independent cause.
worker-slot-heavy behavior	Behavior with high worker-slot time relative to active resource usage. This class is important for queue workers because a task may consume little CPU but hold scarce worker capacity for a long time.
payload-sensitive behavior	Behavior that depends strongly on payload size, input shape, record count, object count, or batch size. Payload-sensitive behavior can still be uncertain if context or hidden state varies.
state-dependent behavior	Behavior that depends on domain-specific or internal system state, such as replica divergence, cache state, compaction debt, segment fragmentation, or dirty data volume.
high-tail-risk behavior	Behavior in which rare slow executions or high-percentile outcomes are operationally important. High-tail-risk behavior is distinct from high average cost.
stable behavior	Behavior with low residual error, low drift, and low variability under comparable execution contexts. Stable behavior may still be expensive.
unstable behavior	Behavior with high variability, drift, residual error, multimodality, or context sensitivity. Unstable behavior is not necessarily high-cost in every attempt.

A.3 Execution Context

Table 5: Execution-context terms.

Term	Definition
execution context	The environment and state under which a task attempt executes. Execution context includes worker state, node pressure, dependency health, network condition, disk pressure, cache state, tenant pressure, retry state, and other runtime or domain-specific conditions.

Term	Definition
context factor	A feature of the execution context that may influence task behavior. Context factors are modeled as influences, not automatically as proven causes.
worker state	The local state of the worker during a task attempt, including local runtime pressure, running work, memory state, garbage-collection pressure, and local scheduling delay.
worker pressure	Pressure on a worker or worker pool, such as busy worker slots, saturated local queues, high utilization, or local scheduling delay.
node pressure	Pressure on the host, virtual machine, container, or node on which the worker runs. Examples include CPU saturation, memory pressure, disk pressure, and network saturation.
dependency health	The operational condition of an external dependency, such as latency, timeout rate, error rate, saturation, quota pressure, or availability.
dependency capacity	The available capacity of an external dependency, such as connection-pool capacity, rate-limit budget, service quota, or downstream throughput.
network condition	The state of the network path relevant to a task attempt, including round-trip time, jitter, packet loss, retransmissions, bandwidth, timeouts, and availability.
network instability	A normalized feature representing unstable network behavior. It may be derived from packet loss, jitter, retransmissions, timeout rate, bandwidth variation, or recent network failures.
replica state	The state of a replica, shard, peer, or replica set involved in a task attempt. Replica state is relevant to replication, repair, synchronization, and consistency tasks.
replica health	A normalized feature representing the operational condition of a replica or peer. Replica health is related to, but distinct from, replication lag.
replication lag	An observation or signal describing how far a replica is behind another source of truth. It may be measured in bytes, records, operations, versions, or time.
disk pressure	Pressure on the storage subsystem, such as queue depth, read latency, write latency, I/O wait, or saturation. Disk pressure is an execution-context factor, not the same as disk I/O demand.
cache state	The condition of a cache relevant to a task attempt, such as warm, cold, stale, evicted, partially populated, or inconsistent.

Term	Definition
cache warmth	A normalized feature representing how prepared or populated a cache is for the task attempt. Cache warmth can influence resource demand and latency.
data locality	The proximity of required data to the worker or execution environment. Examples include local, remote, cross-zone, cross-region, or external storage access.
tenant pressure	Pressure or quota usage associated with a tenant, such as used concurrency, rate-limit consumption, or tenant-level backlog.
retry state	The state of the retry cycle for a task instance or attempt, including attempt number, previous errors, backoff state, retry budget, and retry history.
hidden system dynamics	Unobserved or partially observed system state that affects task behavior, such as internal dependency queues, lock contention, cache eviction history, data fragmentation, noisy neighbors, kernel behavior, or runtime effects. Hidden dynamics are represented through residual error, uncertainty, and reduced confidence when not captured by known features.

A.4 Observations, Signals, and Features

Table 6: Observation, signal, and feature terms.

Term	Definition
observation	Any measurement, event, trace, profile sample, counter, log entry, or domain-specific signal exposed by a queue, worker runtime, operating system, container runtime, dependency, tracing system, or domain component.
raw observation	An unnormalized measurement or event before interpretation. A raw observation does not by itself have stable model meaning.
metric	A time-series, counter, gauge, or histogram observation. Metrics are one kind of observation; traces, logs, profiles, and domain events are observations too.
counter	A metric that counts events, operations, bytes, retries, failures, or other monotonic quantities. A counter may represent a symptom or outcome rather than a root influence.
gauge	A metric representing a current value, such as queue depth, active workers, available connections, or current memory usage.

Term	Definition
histogram	A metric representing a distribution of observed values, such as durations, latencies, sizes, or per-attempt costs.
trace	An execution trace composed of spans. A trace exposes execution structure and timing, but it does not by itself define task-kind-specific relevance, causality, feature mapping, or profiling policy.
span	A timed unit inside a trace representing an operation, processing stage, dependency call, or instrumented region. A span is not the same as a task attempt.
log	A structured or unstructured event record emitted by a system. Logs may expose status, error classes, domain events, or diagnostic information.
runtime profile	A CPU, heap, allocation, lock, blocking, or runtime-level profile. A runtime profile is distinct from a behavior profile.
signal	An interpreted, aggregated, or derived observation that indicates a condition relevant to the model. A signal is an intermediate layer between raw observation and canonical feature.
canonical feature	A typed and normalized model input derived from task metadata, execution context, raw observations, or signals. Task Behavior Graphs operate over canonical features rather than arbitrary raw metrics.
semantic feature	An informal synonym for a feature with explicit semantic meaning. This paper uses the term canonical feature when referring to model inputs with defined type, range, unit, influence domain, and intended meaning.
feature mapping	The process of converting raw observations, signals, task metadata, and context into canonical features. Feature mapping is separate from graph evaluation.
normalization	The process of converting values into a defined type, unit, scale, bucket, or range. For example, network measurements may be normalized into an instability score in $[0, 1]$.
aggregation	The process of combining observations over a time window, group, task kind, attempt set, or distribution. Examples include sum, average, maximum, p95, and p99.
feature registry	A registry of canonical features, including names, types, ranges, units, influence domains, and intended meaning.
metric descriptor	A descriptor of a raw metric, including source, unit, label set, aggregation, cardinality, and mapping to canonical features.

Term	Definition
feature descriptor	A descriptor of a canonical feature, including type, range, unit, semantic meaning, influence domain, and expected use in the model.
cardinality	The number of distinct time series or label combinations produced by a metric. Cardinality affects telemetry overhead and storage cost.
high-cardinality label	A metric label that can create a large or unbounded number of series, such as task id, user id, object id, URL, or unbounded tenant label.
observation window	The time window over which observations are aggregated or interpreted. For example, network instability may be computed over the last 30 seconds.
freshness	How recent and applicable an observation is to the current task attempt. Stale observations may reduce confidence.

A.5 Influence Domains

Table 7: Influence domain terms.

Term	Definition
influence domain	A semantic role assigned to a feature, observation, output, or model concept in the task behavior model. Influence domains are not metric domains: they group model elements by role rather than by telemetry source.
task identity domain	The influence domain containing identifiers and metadata that describe which task is being executed, such as task kind, version, queue, tenant, producer, priority, and deadline.
intrinsic demand domain	The influence domain describing work contained in the task itself, such as payload size, record count, object count, partition size, input format, operation mode, or batch size.
execution context domain	The influence domain describing the environment and state under which the task attempt executes, such as network condition, dependency health, replica state, disk pressure, cache warmth, worker pressure, and retry state.
resource demand domain	The influence domain describing active resource dimensions required or consumed by the task attempt, such as CPU time, memory peak, disk I/O, network I/O, runtime allocations, or accelerator usage. Waiting on dependencies is not active resource demand by itself, although it can contribute to worker occupancy and operational risk.

Term	Definition
worker occupancy domain	The influence domain describing bounded execution capacity held by the task attempt, such as worker-slot time, lease time, connection hold time, lock hold time, semaphore hold time, or tenant concurrency time. Wait-related dimensions such as dependency wait time may contribute to this domain when a task keeps bounded worker capacity while waiting.
outcome domain	The influence domain describing how the task attempt completed, such as success, failure, timeout, cancellation, retryable error, non-retryable error, dead-lettering, or lease expiration.
uncertainty domain	The influence domain describing unreliability or unpredictability in the model estimate or behavior profile, such as residual error, drift, variance, tail ratio, multimodality, missing features, or low sample count.
risk domain	The influence domain describing the consequence-weighted danger of underestimating, misclassifying, or mishandling a task attempt. Examples include queue starvation, retry storm, SLO violation, replication lag, tenant impact, and cascading overload.
profiling observation domain	The influence domain describing what is observed, where it is observed, at what detail level, and under which overhead budget.

A.6 Resource Demand, Occupancy, Cost, and Overhead

Table 8: Resource demand, occupancy, cost, and overhead terms.

Term	Definition
resource demand	The expected or required active resource need of a task attempt along a resource dimension. Resource demand is what the model estimates before or during execution.
resource usage	The observed actual use of a resource during execution. Resource usage is measured after or during execution and is distinct from estimated demand.
active resource usage	Actual use of active resources such as CPU, memory, disk, network, runtime allocation, or accelerator resources. Active resource usage is distinct from waiting and bounded-capacity occupancy.
CPU time	The amount of CPU execution time consumed by a task attempt. CPU time is not the same as wall-clock duration.

Term	Definition
CPU pressure	Pressure on CPU capacity in the worker, container, node, or co-located environment. CPU pressure is an execution-context factor, not the same as task CPU demand.
memory peak	The maximum memory used by a task attempt during execution. Memory peak is a resource-usage or resource-demand dimension.
memory pressure	Pressure on available memory in the worker, container, node, or runtime environment. Memory pressure is an execution-context factor.
allocation behavior	The allocation volume, allocation rate, allocation pattern, or allocation-induced runtime effect of a task attempt. Allocation behavior may influence memory pressure, garbage collection, and CPU time.
disk I/O	Disk reads, writes, operations, bytes, or I/O wait associated with a task attempt. Disk I/O demand is distinct from disk pressure.
network I/O	Network bytes, packets, requests, or transfer volume associated with a task attempt. Network I/O is distinct from network instability.
wait time	Time spent waiting rather than actively consuming a hardware or runtime resource. Wait time may be caused by dependencies, locks, network transfer, backoff, throttling, local scheduling delay, or other blocking conditions.
dependency wait time	Time spent waiting for an external dependency, such as a database, API, remote service, or storage system. Dependency wait time is not active resource demand by itself, but it can increase wall-clock duration and worker occupancy when bounded worker capacity remains held during the wait.
worker occupancy	Bounded execution capacity held by a task attempt over time. Worker occupancy is a first-class cost component in queue-worker systems.
worker-slot time	The time during which a task attempt holds a worker execution slot. This is the core worker-occupancy dimension.
lease time	The time during which a task lease or reservation is held. Lease time may differ from worker-slot time.
connection hold time	The time during which a task attempt holds a connection to a dependency, database, broker, or external service.
lock hold time	The time during which a task attempt holds a lock. Lock hold time can affect other tasks and may contribute to contention.
lock wait time	The time spent waiting to acquire a lock. Lock wait time is distinct from lock hold time.

Term	Definition
semaphore hold time	The time during which a task attempt holds a semaphore, concurrency token, or bounded execution permit.
tenant concurrency time	The time during which a task attempt consumes a tenant-level concurrency slot. This dimension is relevant to fairness and isolation.
wall-clock duration	Elapsed time from attempt start to attempt finish. Wall-clock duration is often related to worker-slot time, but the two are not necessarily identical.
latency	Delay observed by a caller, queue, worker, dependency, or system boundary. Latency is an outcome or symptom, not resource demand by itself.
service time	The time a task attempt spends being served or executed, excluding queue wait time. The exact boundary must be defined for the queue-worker system being analyzed.
queue wait time	The time between enqueue and dequeue or execution start. Queue wait time is distinct from worker-slot time.
task cost	An umbrella term for resource demand, worker occupancy, and execution overhead associated with a task attempt. Operational risk is not part of task cost.
execution overhead	Overhead inherent to executing a task attempt, such as serialization, input fetching, setup, acknowledgement, commit, cleanup, or retry bookkeeping.
profiling overhead	Overhead caused by profiling and instrumentation, including tracing, metrics, runtime profiling, high-cardinality labels, sampling decisions, and telemetry export.
telemetry overhead	CPU, memory, allocation, network, storage, or cardinality cost introduced by telemetry collection and export.

A.7 Uncertainty, Confidence, Residuals, and Risk

Table 9: Uncertainty, confidence, residual, and risk terms.

Term	Definition
uncertainty	Unreliability or unpredictability of a task behavior estimate. Uncertainty may come from missing features, hidden state, measurement error, drift, heavy tails, multimodality, or insufficient history.
entropy	A possible measure or family of measures for unpredictability. The term should only be used after specifying what distribution, signal, or behavior process is being measured.

Term	Definition
confidence	The degree of trust in an estimate, behavior profile, graph edge, feature mapping, or model assumption. Confidence is distinct from uncertainty.
residual error	The part of observed behavior not explained by the current model estimate. Residual error exposes missing features, hidden influences, drift, or model mismatch.
prediction error	The difference between predicted and observed value for a resource, occupancy, duration, outcome, or risk-relevant dimension.
drift	A change in task behavior, feature mapping, graph relationship, context, or profile over time. Drift is a source of uncertainty and reduced confidence.
variance	The statistical spread of observed values. Variance is a variability signal but is not identical to entropy.
tail ratio	A ratio comparing a tail quantile with a central quantile, such as p95/p50 or p99/p50. Tail ratios help identify high-tail-risk behavior.
multimodality	The presence of multiple behavioral regimes or modes in the observed behavior of a task kind or task family.
operational risk	The consequence-weighted danger of underestimating, misclassifying, or mishandling a task attempt. Operational risk is separate from task cost.
consequence	The severity of the effect caused by an incorrect estimate or control decision, such as delayed processing, queue starvation, SLO violation, replication lag, or cascading overload.
blast radius	The scope affected by an error or overload event, such as a single task, queue, worker pool, tenant, replica set, dependency, cluster, or control plane.
SLO violation risk	The operational risk that a task attempt, task kind, queue, tenant, or service will violate a service-level objective.
queue starvation risk	The risk that long-running or high-occupancy tasks delay or prevent other tasks from receiving execution capacity.
retry amplification risk	The risk that retries increase work, dependency pressure, queue occupancy, or worker occupancy enough to worsen overload.
cascade overload risk	The risk that local overload propagates to other queues, workers, dependencies, tenants, or control loops.

A.8 Task Behavior Graphs

Table 10: Task Behavior Graph terms.

Term	Definition
Task Behavior Graph	A typed graph associated with a task kind or task family. It connects canonical features across influence domains and supports estimation of resource demand, worker occupancy, uncertainty, and operational risk. A Task Behavior Graph is not assumed to prove causality by itself; it makes modeled influence assumptions explicit.
graph node	An element of a Task Behavior Graph representing a canonical feature, output, dimension, intermediate signal, or model concept. Each node belongs to one or more influence domains.
graph edge	A modeled influence relationship between two graph nodes. A graph edge is not a proven causal link unless validated by additional evidence.
influence role	The semantic role of a node or edge in the graph, such as baseline demand, contextual amplification, additive overhead, symptom, outcome link, risk propagation, uncertainty indicator, or profiling trigger.
baseline demand edge	An edge that connects an intrinsic feature to a baseline resource-demand or occupancy estimate. For example, payload size may contribute to network I/O or disk I/O baseline demand.
contextual amplification edge	An edge by which an execution-context factor amplifies or suppresses demand, occupancy, uncertainty, or risk.
additive overhead edge	An edge that represents additional fixed or variable overhead, such as setup, serialization, acknowledgement, commit, or retry bookkeeping.
symptom edge	An edge that connects an influence to a symptom or observed effect, such as network degradation contributing to retries or longer wall-clock duration.
outcome link	An edge that connects modeled behavior to an outcome, such as timeout, failure, success, cancellation, or dead-lettering.
risk propagation edge	An edge that connects demand, occupancy, uncertainty, or outcome to an operational-risk dimension.
uncertainty indicator	A node or edge role indicating that a signal changes uncertainty or confidence rather than directly changing resource demand.
sensitivity	The estimated strength of a relationship between graph nodes. Sensitivity may be expert-defined, learned, calibrated, or represented as a qualitative level.

Term	Definition
relevance	Whether a feature or relationship matters for a task kind, task family, or behavior profile. Relevance is distinct from sensitivity.
graph evaluation	The process of applying a Task Behavior Graph to canonical features and execution context in order to estimate demand, occupancy, uncertainty, residuals, and risk.
graph template	A reusable graph structure that can be specialized for multiple related task kinds within a task family.
model assumption	An explicit assumption encoded in the graph, feature mapping, or policy. Model assumptions should be validated, calibrated, or revised when observations contradict them.

A.9 Profiling and Observation Policy

Table 11: Profiling and observation-policy terms.

Term	Definition
profiling	The process of collecting, interpreting, and using observations to estimate task behavior. In this paper, profiling is broader than CPU profiling.
task profiling	Profiling logical task attempts in queue-worker systems in order to estimate resource demand, worker occupancy, uncertainty, and operational risk.
observation strategy	A decision about what to observe, where to observe it, how often to observe it, and under what overhead budget.
profiling policy	The selected observation and instrumentation strategy for a task kind, task attempt, or execution context.
accounting	Recording basic execution facts, such as start time, finish time, status, duration, input size, output size, resource usage, or occupancy.
minimal accounting	A low-overhead accounting level that records only essential task facts, such as start time, finish time, status, and basic size information.
detailed accounting	An accounting level that records more detailed resource, occupancy, stage, dependency, and outcome information.
tracing	Capturing execution structure and timing through traces and spans. Tracing is an observation method, not a behavior model by itself.
sampled tracing	Tracing only a subset of tasks, attempts, or traces in order to control overhead.

Term	Definition
tail-aware profiling	A profiling strategy that preferentially observes slow, failed, high-risk, or tail-heavy attempts.
diagnostic profiling	Deep and usually expensive profiling used to investigate unexplained behavior, high residual error, drift, or model failure.
isolated execution profiling	Profiling in a more controlled or isolated execution environment in order to reduce interference or better attribute behavior.
cooperative instrumentation	Instrumentation in which worker or task code explicitly emits structured observations such as spans, timers, byte counts, record counts, dependency calls, stage boundaries, and domain-specific signals.
sampling	Selecting a subset of tasks, attempts, events, spans, traces, or profile samples for observation or retention.
head sampling	A trace-sampling strategy that decides whether to sample before the full trace is observed.
tail sampling	A trace-sampling strategy that decides whether to sample after seeing all or most of a trace.
observation point	A point in the task lifecycle where observations can be collected, such as enqueue, dequeue, reserve, start, dependency call, retry, commit, acknowledge, or finish.
profiling budget	The allowed overhead for profiling and telemetry, including CPU, memory, allocation, storage, network, and cardinality costs.
hot path	The latency- or throughput-sensitive path of task execution where added instrumentation overhead can affect production behavior.

A.10 Manifests, Registries, and Domain Packs

Table 12: Manifest, registry, and domain-pack terms.

Term	Definition
Task Behavior Manifest	A declarative description of the behavior model for a task kind or task family. It may describe features, domains, graph nodes, graph edges, observation requirements, profiling policy defaults, known gaps, and versioning.
manifest	A machine-readable or human-readable specification used to declare part of the task behavior model.
domain pack	An extension package that provides domain-specific metrics, feature mappings, feature descriptors, graph templates, examples, or profiling defaults.

Term	Definition
metric descriptor registry	A registry describing raw metrics and how they can be interpreted, aggregated, and mapped to canonical features.
graph declaration	The part of a manifest that declares graph nodes, graph edges, influence roles, sensitivities, confidence, and applicability conditions.
profiling policy declaration	The part of a manifest that declares default observation strategies, required observation points, sampling rules, and profiling overhead limits.
schema	A machine-validation structure for manifests, descriptors, registries, domain packs, or graph declarations.
versioning	The practice of assigning versions to schemas, manifests, domain packs, task kinds, feature registries, or graph structures in order to manage evolution and compatibility.
compatibility	The ability of manifests, schemas, domain packs, task versions, and graph structures to work together without semantic or validation conflicts.
coverage	The set of task kinds, domains, features, metrics, observation points, or risks covered by a model, manifest, or domain pack. Coverage is not the same as global completeness.
known gap	A missing, unavailable, weak, or intentionally unsupported feature, observation, domain, or graph relationship. Known gaps should remain visible rather than being hidden inside the model.
residual visibility	The ability of the model to expose unexplained behavior through residual error, reduced confidence, or missing-feature indicators.
canonical namespace	A stable naming scheme for canonical features, resource dimensions, occupancy dimensions, and risk dimensions, such as `context.network.instability` or `occupancy.worker.slot_time`.

A.11 Queue-Worker Lifecycle Terms

Table 13: Queue-worker lifecycle terms.

Term	Definition
enqueue	The lifecycle point at which a task instance is submitted to a queue.
dequeue	The lifecycle point at which a worker takes a task instance from a queue for processing.

Term	Definition
reserve	The lifecycle point at which a worker or worker framework claims a task instance, often through a lease, visibility timeout, or reservation mechanism.
start	The lifecycle point at which a task attempt begins execution.
input fetch	The stage in which the worker fetches external input, payload data, referenced objects, or domain state required by the task attempt.
deserialize	The stage in which the worker decodes, parses, or prepares the task payload or input data for processing.
processing stage	A logical phase inside task execution, such as validation, transformation, dependency call, write, commit, or cleanup.
dependency call	An interaction with an external service, API, database, storage system, broker, or remote dependency during task execution.
retry	A repeated execution attempt or repeated operation after failure, timeout, lease expiration, cancellation, or retryable error.
backoff	A delay before retrying a task attempt or operation. Backoff can contribute to worker occupancy if execution capacity remains held.
output write	The stage in which task output, state changes, generated data, or side effects are written.
commit	The stage in which task effects are finalized, persisted, checkpointed, or made visible.
acknowledge (ack)	The lifecycle point at which a worker confirms task completion to the queue or broker.
discard	The lifecycle outcome in which a task instance or attempt is intentionally dropped or ignored.
dead-letter	The lifecycle outcome in which a task is moved to a dead-letter queue or equivalent failure storage.
finish	The lifecycle point at which a task attempt ends, whether by success, failure, timeout, cancellation, or another terminal outcome.
lease expiration	The lifecycle event in which a task reservation expires before successful acknowledgement or completion.
cancellation	The lifecycle event in which a task attempt is explicitly stopped before normal completion.